

INFRASTRUCTURE AS CODE · AWS · TERRAFORM

Project Genesis

A production-ready, containerized AWS deployment — Terraform-managed infrastructure, a FastAPI backend, Dockerized containers, and Git-driven workflows. Built for scalability, security, and repeatable provisioning.



What Is Project Genesis?



Terraform IaC

Modular AWS infrastructure — VPC, subnets, ALB, ASG, RDS — fully version-controlled



FastAPI Backend

Python REST API with PostgreSQL, Pydantic models, and external API integration



Dockerized Deploy

Containerized via Docker Compose with runtime secret injection from SSM



Git Workflow

EC2 instances auto-clone the repo at bootstrap; all changes tracked and auditable

FastAPI Backend Application



Core Capabilities

- RESTful endpoints with Pydantic request/response validation
- SQLAlchemy ORM connected to PostgreSQL RDS
- Full CRUD operations for deal management
- External integration with CheapShark gaming deals API
- Environment-driven config via `DATABASE_URL`

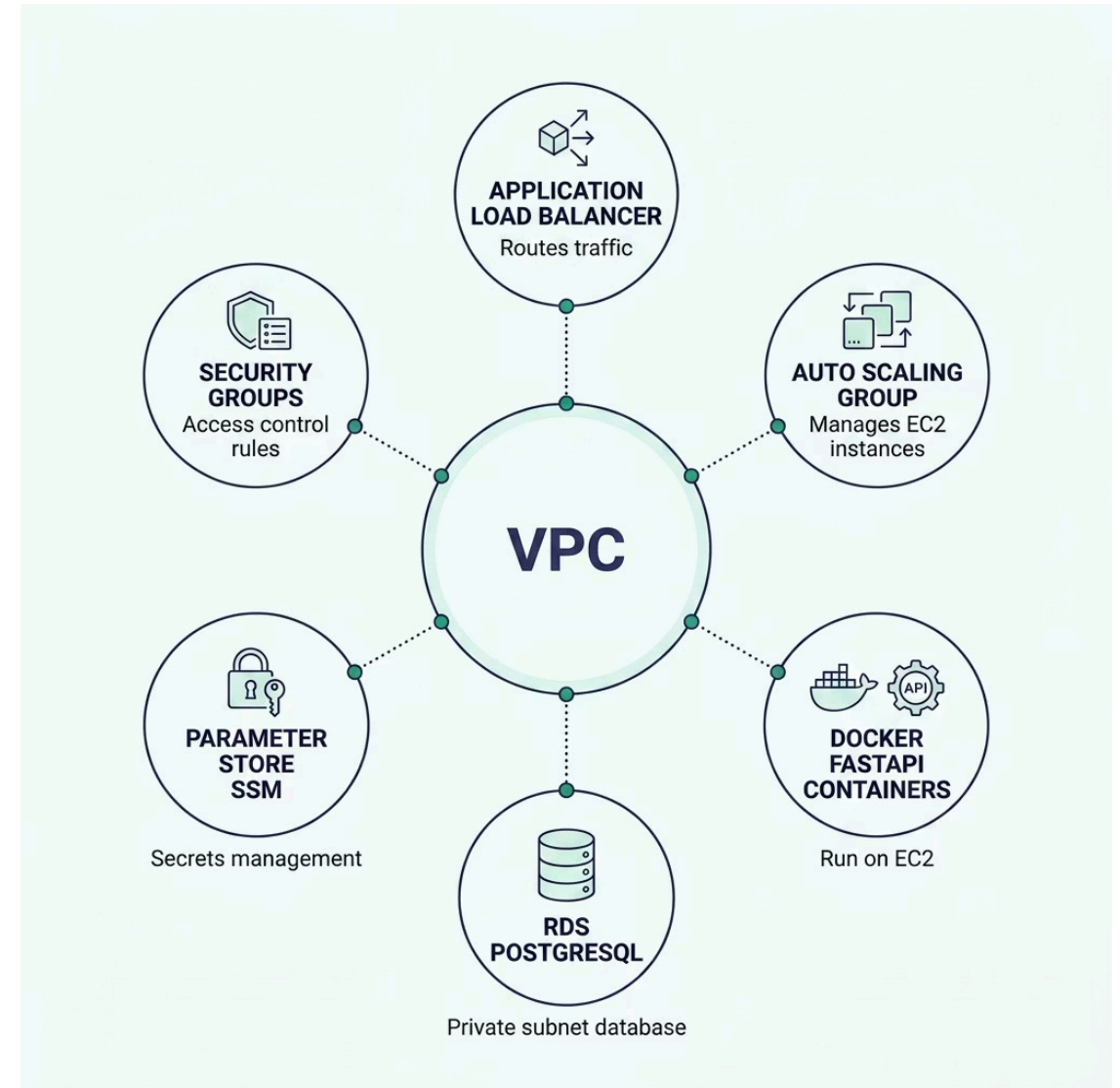
API Endpoints

- `/health` — ALB health check probe
- `/configs` — Runtime configuration
- `/deals` — Live gaming deals from CheapShark
- `/saved-deals` — Persisted deals in PostgreSQL

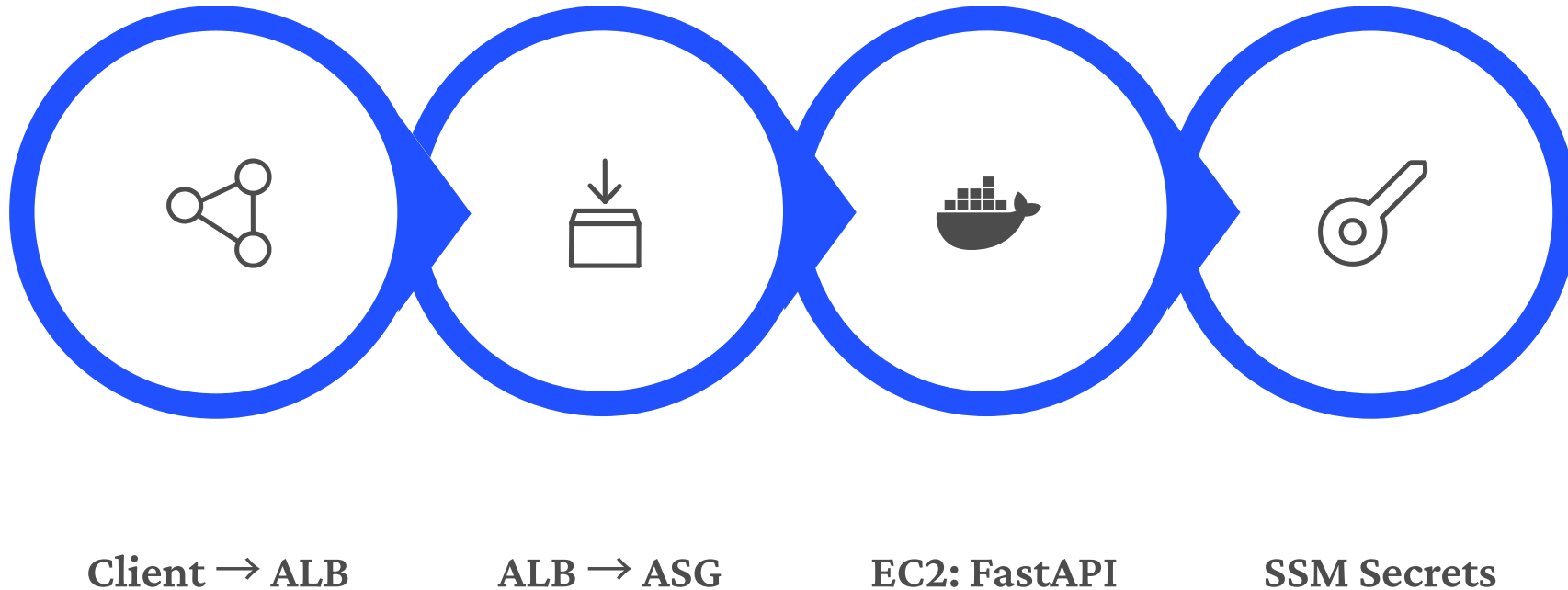
AWS Cloud Architecture

Project Genesis spans a multi-tier AWS architecture — isolated networking, managed compute, and secure data storage — all provisioned through Terraform modules.

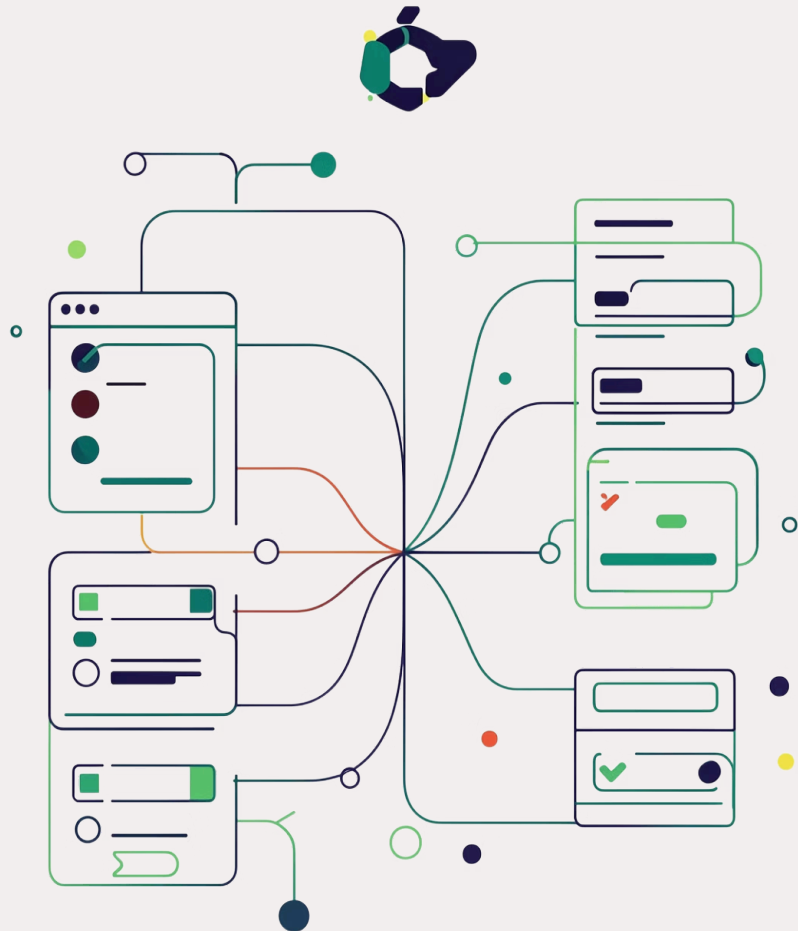
- **VPC** with public/private subnets across AZs
- **ALB** distributes traffic to healthy targets
- **Auto Scaling Group** with Launch Template
- **RDS PostgreSQL** in private subnets
- **SSM Parameter Store** for secrets management
- **Security Groups** enforcing least-privilege access



Request Flow Through the Stack



Traffic enters through the ALB, which performs health checks against `/health` and routes only to healthy targets. EC2 instances run the containerized FastAPI app, connecting securely to RDS PostgreSQL in private subnets. Secrets are injected at runtime via SSM – never stored in code or environment files.



VERSION CONTROL

GitHub Integration & Workflow

Repo-Driven Bootstrap

EC2 instances clone the repository automatically during launch via `user_data`. Infrastructure and application code are both version-controlled — every change is tracked, reviewable, and reversible.

Standard Git Workflow

```
git clone https://github.com/your-repo/genesis.git
git checkout -b hw24
git add .
git commit -m "Add new feature"
git push origin hw24
```

Command Line Execution Flow

Terraform — Provision Infrastructure

```
terraform init  
terraform plan  
terraform apply
```

Docker — Start Application

```
docker-compose up -d --build  
docker ps
```

Verify — Health Check

```
curl http://ALB_DNS/health
```

Execution Sequence

1. Initialize Terraform and download provider plugins
2. Review the execution plan before applying changes
3. Provision VPC, ALB, ASG, RDS, and SSM parameters
4. EC2 instances launch and run `docker-compose up -d`
5. FastAPI container starts; ALB health checks pass
6. Validate live endpoint via `curl`

Golden AMI: Solving Slow Boot Times

✗ Before Golden AMI

Every EC2 launch triggered a fresh install of Docker, Git, and AWS CLI – adding several minutes to boot time and introducing inconsistency across instances.

- Slow Auto Scaling response
- Non-deterministic instance state
- Failed health checks during install

✓ After Golden AMI (HW24)

A pre-baked AMI ships with all dependencies pre-installed. Instances are ready to serve traffic almost immediately after launch.

- Docker, Git, AWS CLI pre-installed
- Faster Auto Scaling and instance refresh
- Truly immutable, consistent infrastructure

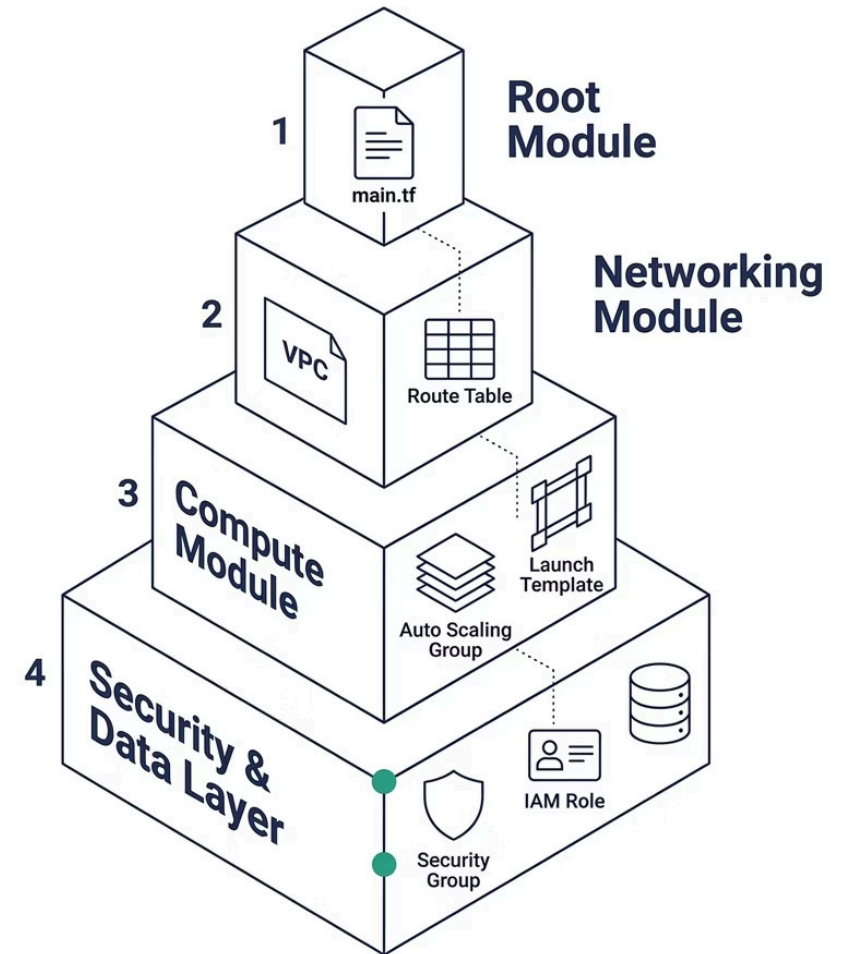
Terraform Structure & Security

Module Layout

- `main.tf` — Root module, orchestrates all components
- `networking/` — VPC, subnets, route tables, IGW
- `compute/` — ASG, Launch Template, `user_data`

Security Controls

- RDS PostgreSQL isolated in private subnets
- Security Groups enforce least-privilege access
- Secrets stored in SSM Parameter Store
- EC2 uses scoped IAM roles — no hardcoded credentials





RESULTS

DevOps Best Practices & Final Results



Infrastructure as Code

Reusable Terraform modules with remote state for consistent, auditable provisioning



Immutable Servers

Golden AMI + instance refresh — no in-place changes, always reproducible



Containerized Deployment

Docker Compose with environment-driven config and SSM secret injection



Git-Based Workflow

Every infra and app change tracked, reviewed, and deployed through Git



Project Genesis delivers: Faster deployments · Rapid scaling · Production-ready architecture · Secure backend · Automated provisioning

Kubernetes Fundamentals

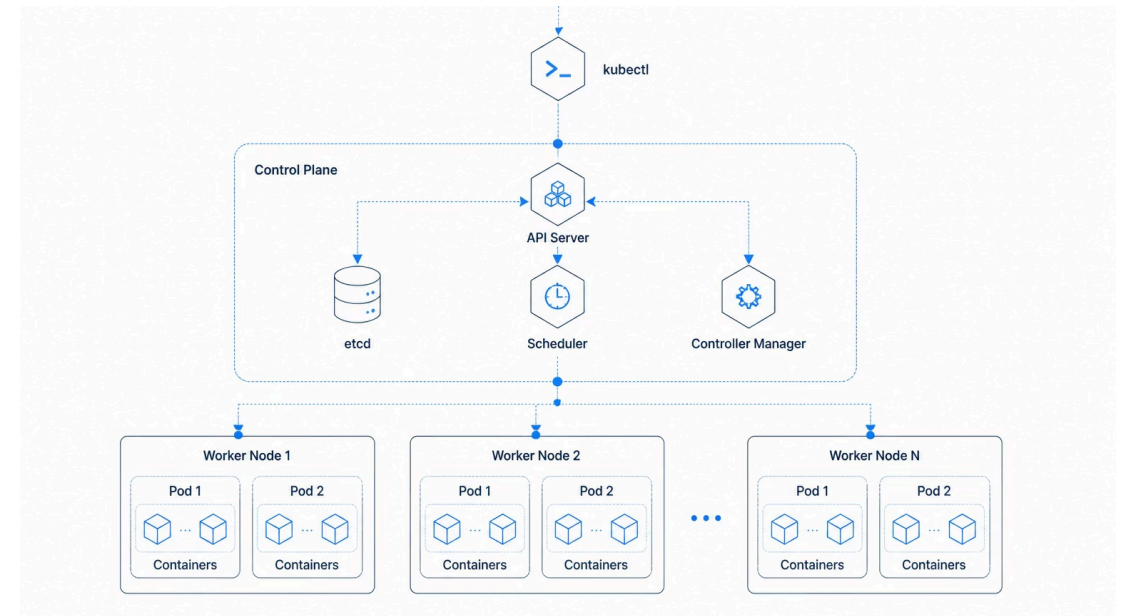
Why Kubernetes?

- Automates container deployment across clusters
- Self-healing: restarts failed containers automatically
- Intelligent scaling: adds/removes instances based on demand
- Service discovery & load balancing built-in
- Rolling updates with zero downtime

Without Kubernetes

- Manual Docker container management
- Manual restart on failures
- Complex scaling logic
- Network routing challenges

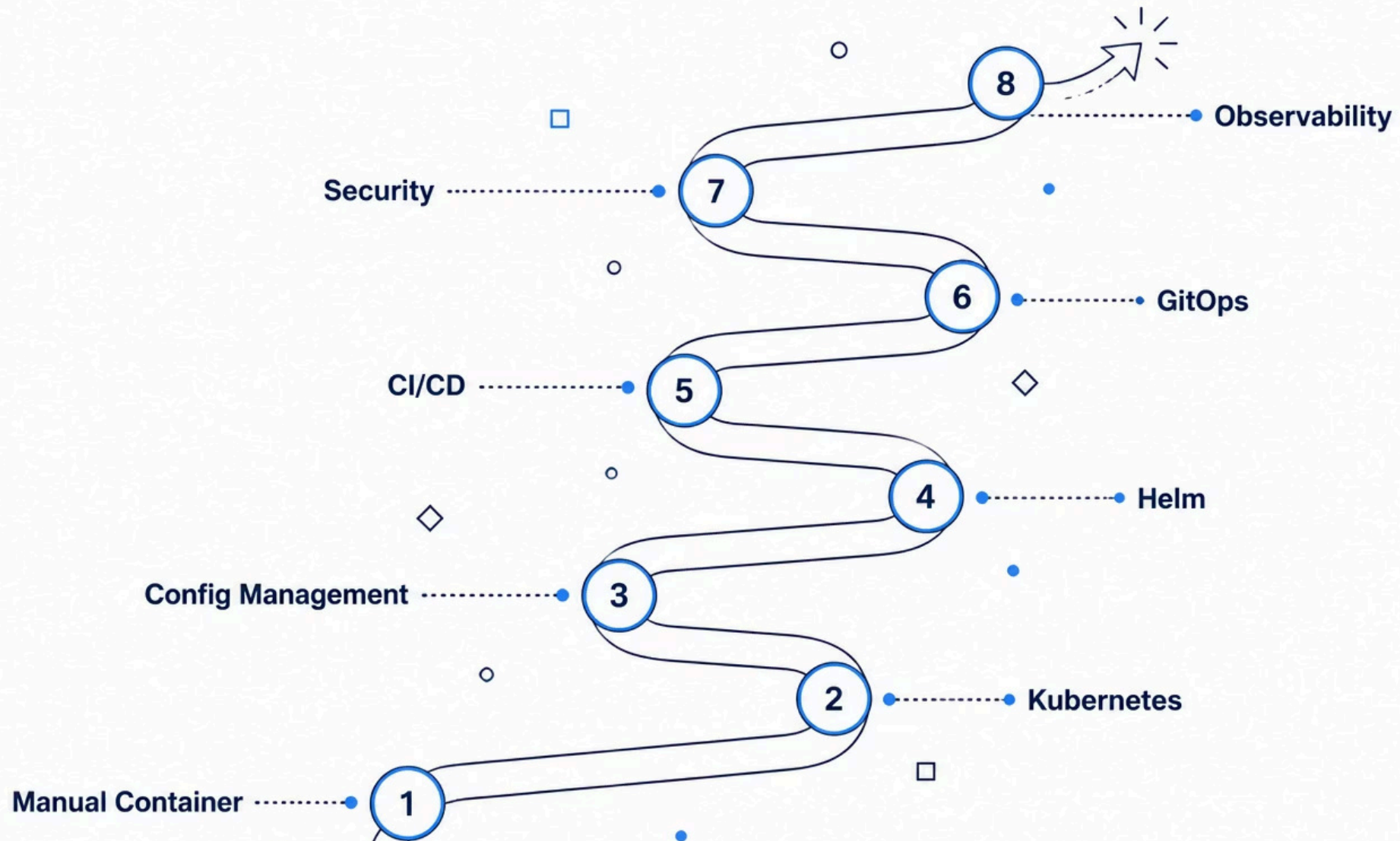
Kubernetes continuously ensures the desired state of your applications



From Bare Metal to Full Observability

The Project Genesis DevOps Journey

An app's evolution from manual deployment to a self-healing, observable production system on Kubernetes



Kubernetes Fundamentals: Deployment, Service, Ingress

The Problem: Fragile Manual Infrastructure

- Apps on bare EC2/VMs require manual restarts when they crash
- Scaling up means manually launching new instances
- No built-in load balancing or service discovery
- Every failure is a page at 3 AM

Kubernetes: Declarative Desired-State Orchestration



Deployment

Self-healing replica management (if a pod dies, K8s restarts it)



Service

Stable network identity (DNS name that doesn't change)



Ingress

External HTTP routing (single entry point for traffic)

Kill a pod, watch it come back in seconds — the system self-heals without human intervention.

Configuration & Secrets Management

The Problem: Hardcoded Config & Passwords

- Database passwords baked into Docker images
- API keys committed to Git
- Different config needed per environment (dev/staging/prod)
- Rebuilding images just to change a setting

Externalizing Configuration



ConfigMap

Non-sensitive config (API endpoints, feature flags, log levels)



Secret

Sensitive data (passwords, tokens, API keys) – base64 encoded at rest



PersistentVolumeClaim (PVC)

Stateful storage that survives pod restarts

Why This Matters: 12-Factor Apps

Configuration should be environment-specific, not baked into the code. This enables the same Docker image to run unmodified across development, staging, and production environments, improving consistency and reducing deployment risks.

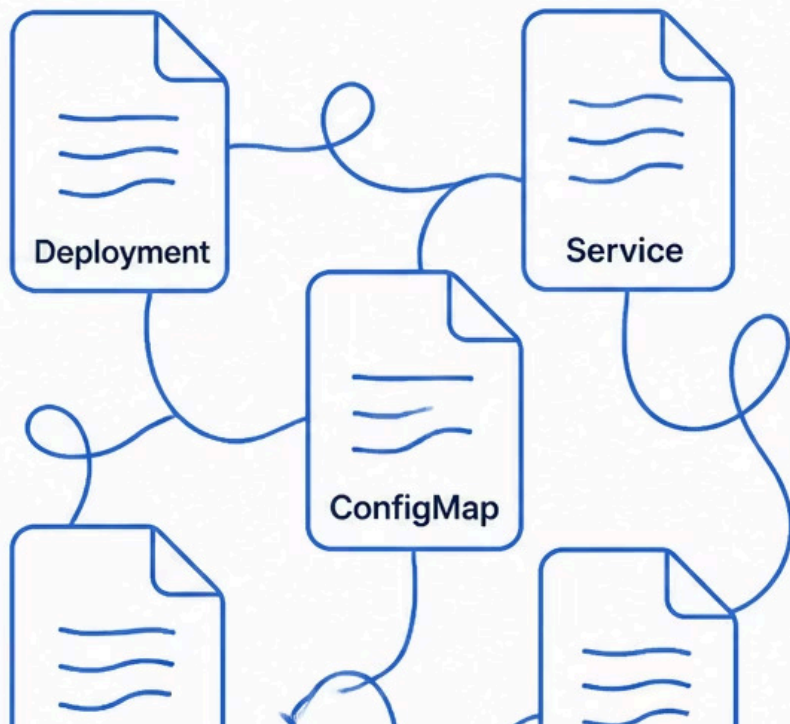
Helm: The Package Manager for Kubernetes

The Problem: YAML Sprawl

- 6+ raw YAML files per app (Deployment, Service, ConfigMap, Secret, Ingress, etc.)
- No reuse across environments (copy-paste for dev vs prod)
- No versioning or rollback capability
- Hard to share apps with the team or community

Before Helm

chaotic cluster of scattered,
individual YAML files

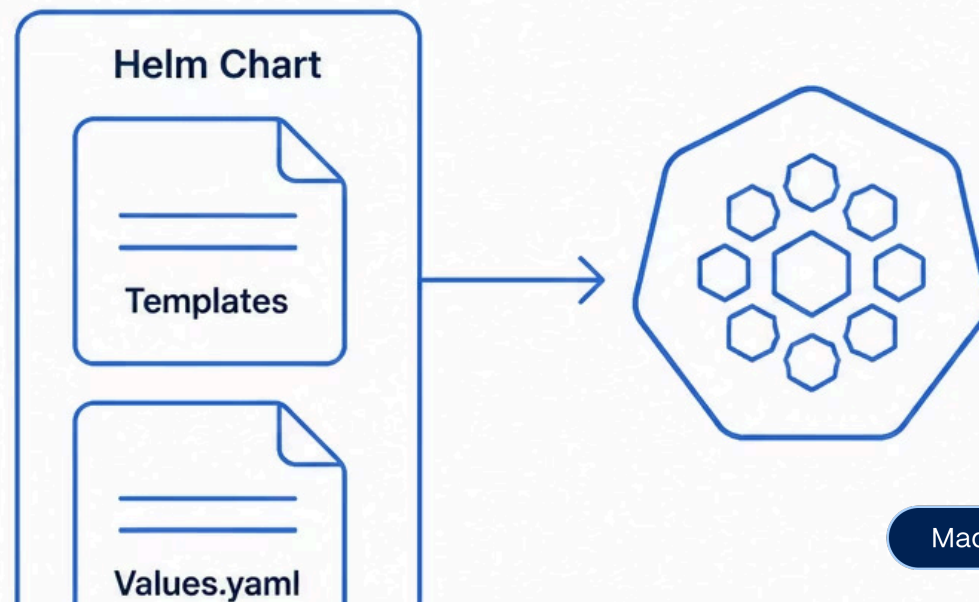


Helm: Templates + Package Management

1. **Chart** – A packaged, versioned, installable Kubernetes app (like npm packages or apt)
2. **Values.yaml** – Single source of truth for all configurable knobs (replicas, image tag, resources)
3. **Templates** – YAML files with placeholders that get filled in from values
4. **Helm upgrade/rollback** – Change values and upgrade, or instantly rollback to previous version

After Helm

single, neatly organized Helm Chart
for simplified packaging and management

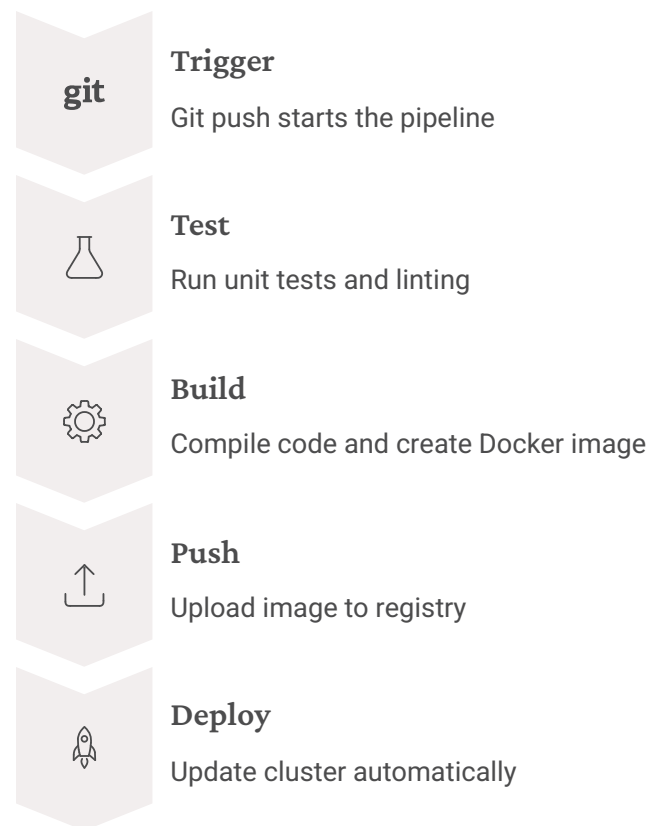


CI/CD Foundations: Automating Build & Deploy

The Problem: Manual Deployments

- Developers manually run build and push commands
- Easy to forget steps or push wrong image versions
- No automated testing before deployment
- Deployments are unpredictable and hard to track

CI/CD Pipeline: Automated Assembly Line



One git push triggers the entire pipeline automatically.

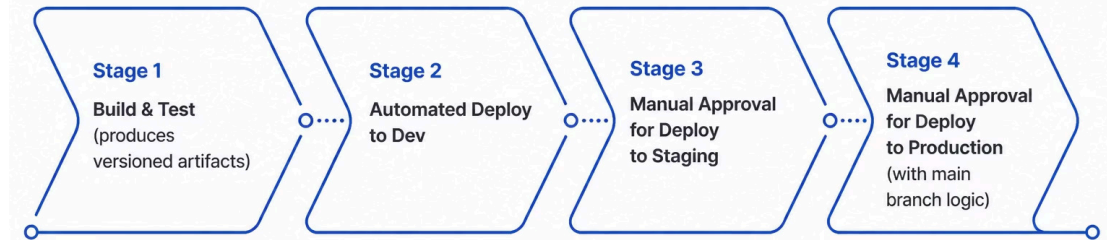
Self-hosted runners give you full control over the execution environment.

Multi-Stage Pipelines: Real-World Delivery

The Problem: Simple Pipelines Aren't Enough

- A single test-build-push job doesn't model real software delivery
- Different environments need different rules (auto-deploy to dev, manual approval for prod)
- Need to distinguish between build artifacts and cached dependencies
- Branch-based rules: feature branches test only, main branch deploys

Multi-Stage Pipeline Architecture



1. **Artifacts vs Cache** — Artifacts are outputs you keep (e.g., Docker image), cache speeds up rebuilds
2. **Ordered Stages** — Each stage depends on the successful completion of the previous one
3. **Environment Rules** — Dev auto-deploys, staging needs approval, prod requires manual gates
4. **Branch Logic** — Different pipeline paths for feature branches vs. main branch deployments

❏ **A real pipeline is like a factory assembly line:** each station depends on the previous one, and quality gates prevent bad products from moving forward.

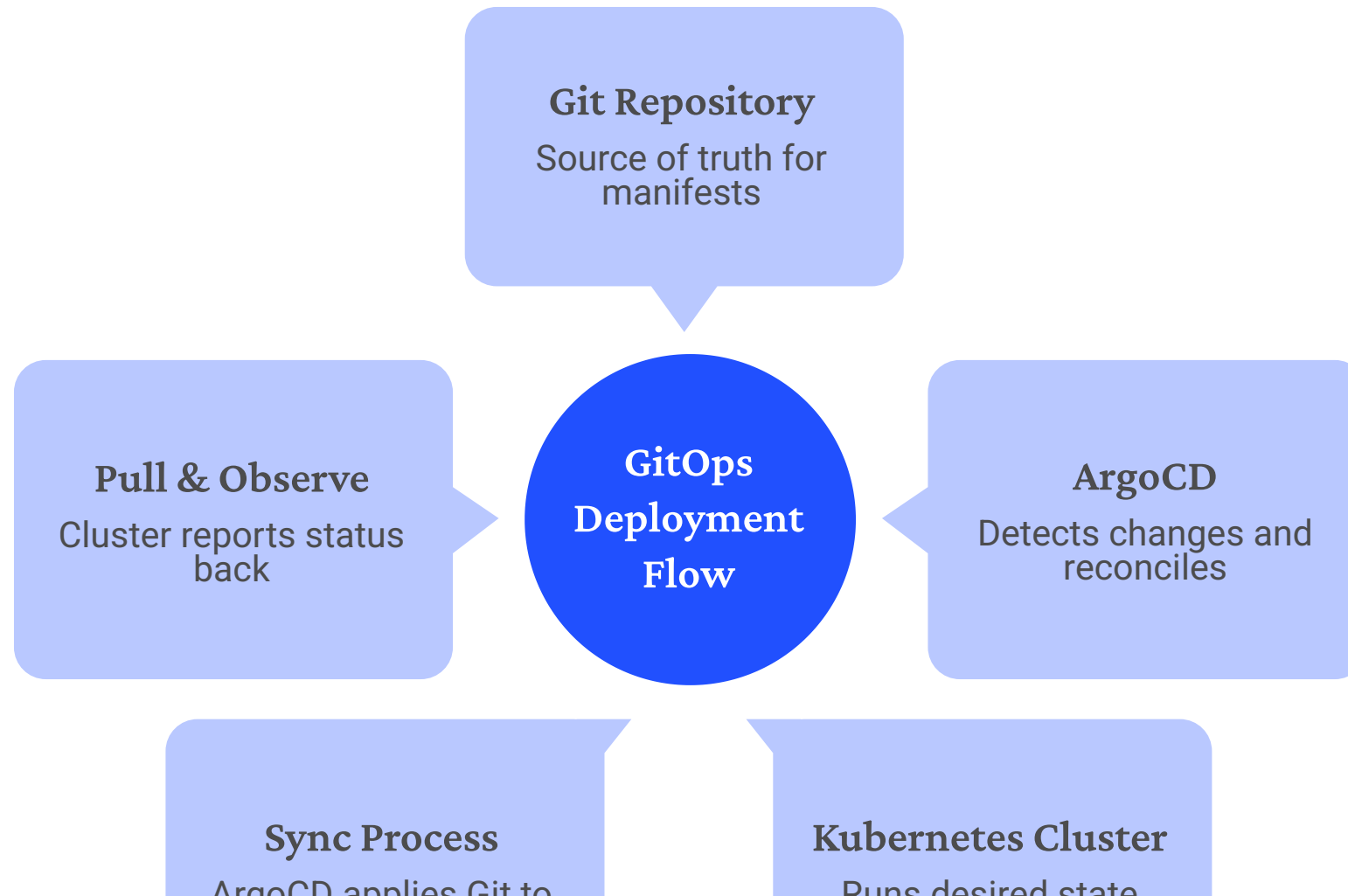
GitOps with ArgoCD: The Self-Healing Cluster

The Problem: Push-Based Deployments

- CI/CD pipelines push directly into the cluster (requires cluster credentials in pipeline)
- Manual kubectl changes drift the cluster away from Git
- No audit trail of who changed what in the cluster
- Cluster state and Git state can diverge silently

GitOps: Git as Single Source of Truth

1. **Pull Model** — ArgoCD controller PULLS from Git, not pushed to by pipeline
2. **Sync** — ArgoCD continuously compares Git state vs cluster state
3. **Self-Heal** — If someone manually changes the cluster, ArgoCD reverts it back to Git
4. **Prune** — Deletes resources from cluster if they're removed from Git



Security Gates: Shift Left with Trivy

The Problem: Vulnerable Images Ship to Production

- Pipelines build and push whatever image you created
- Vulnerabilities discovered after deployment
- Security logic copy-pasted across every repo (not DRY)
- A security gate only counts if it can actually fail the pipeline

Trivy: Automated Vulnerability Scanning

1. **Shift Left** — Scan images during build, not after deployment
2. **Trivy Scanner** — Scans Docker images for known CVEs
3. **Fail the Pipeline** — If vulnerabilities exceed threshold, block the push
4. **DRY Principle** — Shared templates and reusable workflows prevent copy-paste

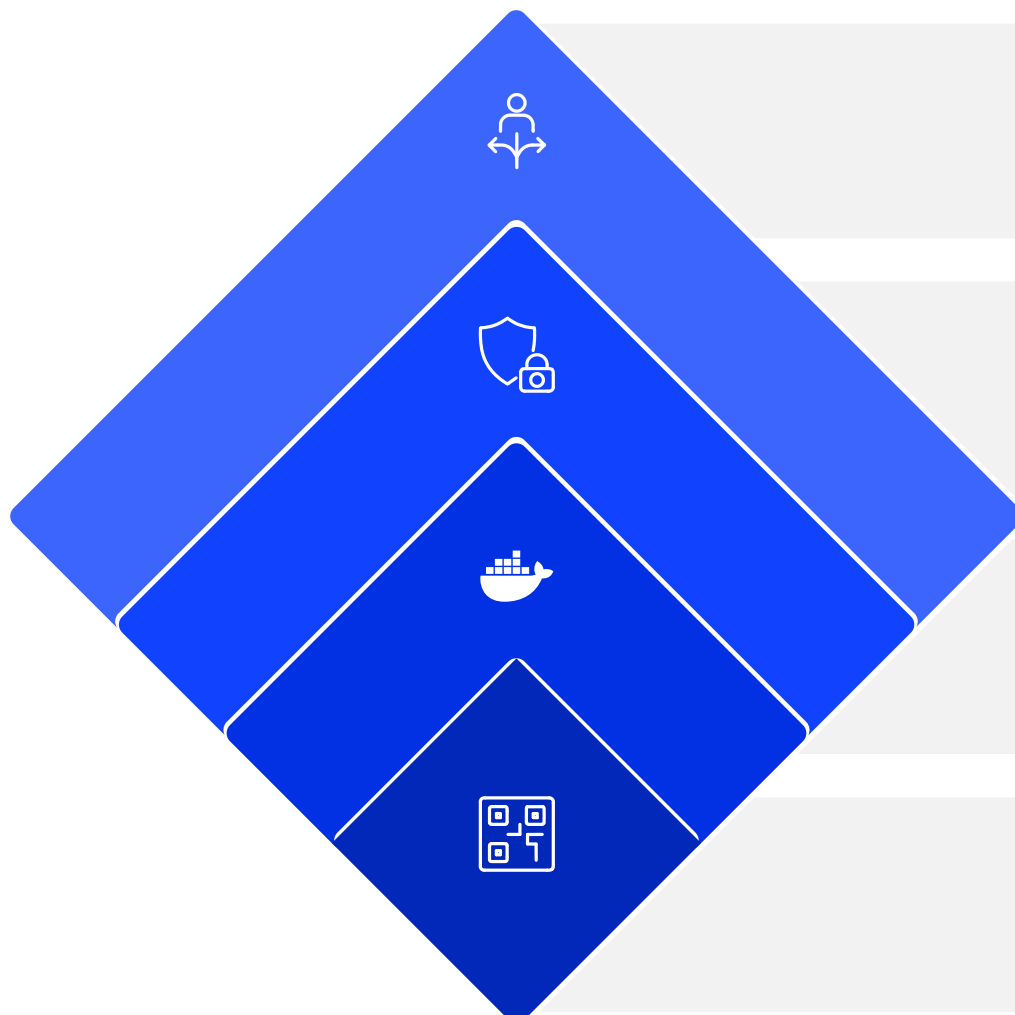


Image Push?

Allow or block based on scan

Trivy Scan

Gate: scan for vulnerabilities

Build Image

Containerize application

Code Commit

Push changes to repo

Observability Pillar 1: Metrics (Prometheus + Grafana)

The Problem: Flying Blind

- The system runs, but you have no idea if it's healthy
- Is it struggling under load? Are requests slow?
- No visibility into CPU, memory, request latency, error rates
- Can't make data-driven decisions about scaling or optimization

Prometheus + Grafana: Metrics & Dashboards

1. **Prometheus** — Scrape-based metrics collection, pulling data from applications every 15 seconds.
2. **ServiceMonitor** — A Kubernetes Operator that automatically discovers services to scrape for metrics.
3. **Grafana** — A powerful open-source platform for visualizing and analyzing collected metrics in customizable dashboards.
4. **RED Method** — Key metrics for monitoring: Rate (requests/sec), Errors (error rate), Duration (latency).

App Emits Metrics

Prometheus Scrapes

Grafana Dashboards

This diagram illustrates the flow from your application generating raw data to it being transformed into actionable insights on a dashboard.



Dashboards as code: Commit dashboard JSON to Git so your monitoring evolves alongside your application's features.

The RED method provides a straightforward mental model for identifying critical metrics to measure.

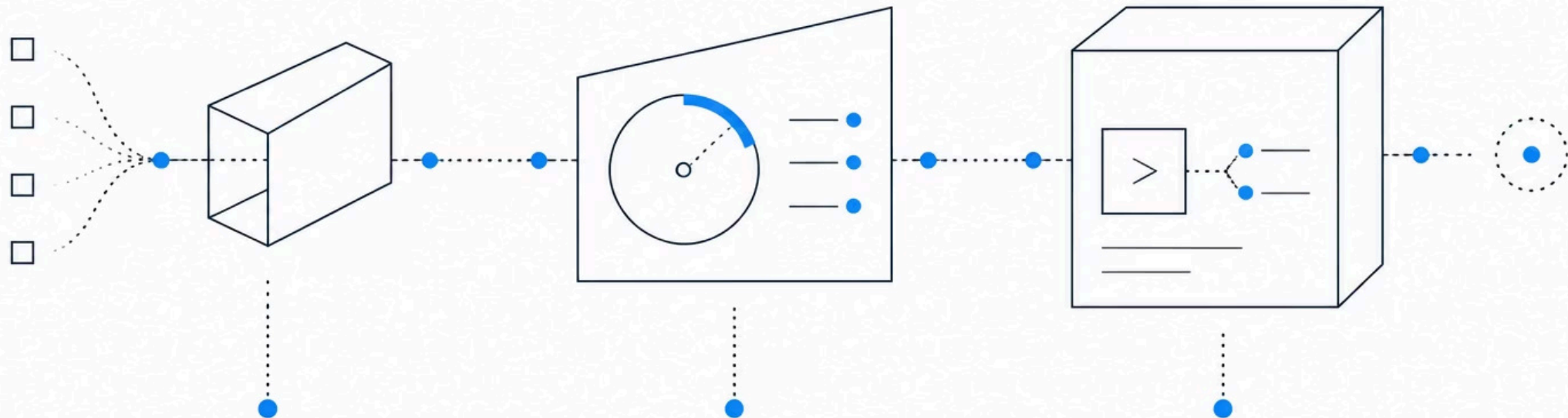
Observability Pillar 2: Alerting (SLOs + PrometheusRule)

The Problem: Dashboards Require Watching

- Dashboards require someone to be staring at them 24/7
- By the time you notice a problem, users are already affected
- Need automated alerts that notify you when things break
- Alert fatigue from too many false positives

SLOs & PrometheusRule: Intelligent Alerting

1. **SLO (Service Level Objective)** — Pick a target just high enough (e.g., 99.9% uptime)
2. **Burn Rate** — How fast you're consuming your error budget
3. **PrometheusRule** — Define alert conditions based on metrics
4. **For Clause** — Wait N minutes before firing alert (prevents flapping)



A metrics stream
feeding into an
SLO calculation

**Showing remaining
error budget,**
which then feeds
into a Prometheus Rule

**That defines a
burn rate threshold,**
triggering an alert if
the burn rate is exceeded

Observability Pillar 3: Logging & Tracing

The Problem: Alerts Tell You Something Is Wrong, Not Why

- An alert fires: "Error rate is high"
- But why? Which requests are failing?
- Logs are scattered across pods and disappear when pods restart
- Can't follow a single request through a distributed system

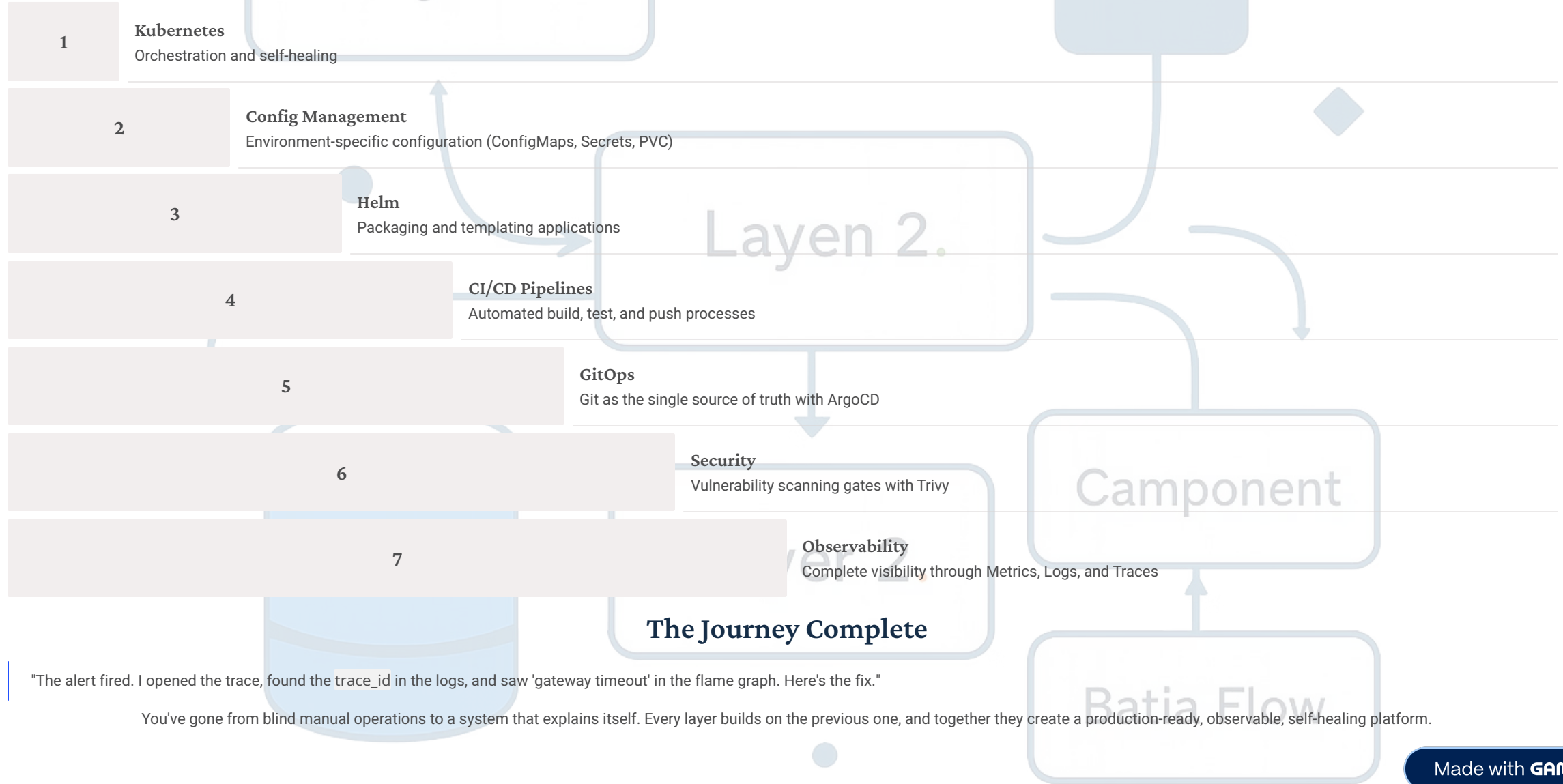
Structured Logs + Distributed Tracing

1. **Structured Logs (JSON)** — Searchable, parseable logs instead of free text
2. **Central Log Shipping** — Fluent Bit sends logs to Loki/EFK (logs outlive pods)
3. **Distributed Tracing** — OpenTelemetry + Jaeger follows one request across services
4. **Golden Thread** — trace_id ties a single log line to a full flame graph



From Bare Metal to Full Observability: The Complete Stack

Building a robust, secure, and self-healing cloud-native platform.



"The alert fired. I opened the trace, found the trace_id in the logs, and saw 'gateway timeout' in the flame graph. Here's the fix."

You've gone from blind manual operations to a system that explains itself. Every layer builds on the previous one, and together they create a production-ready, observable, self-healing platform.

Kubernetes Workflow

Deployment Lifecycle

1. Developer runs: `kubectl apply -f deployment.yaml`
2. API Server validates the request
3. etcd stores the desired cluster state
4. Scheduler selects the best Worker Node
5. kubelet starts the Pod on the node
6. kube-proxy routes traffic to the Pod
7. Controller Manager continuously monitors and self-heals

